

개인 포트폴리오

메이플스토리 : 메르세데스

Win32 · GDI+ 로 재현한 액션 RPG 프로토타입



박 찬 욱

서울게임아카데미 · 1주 개인 포트폴리오

1. 개요

원작 게임 **메이플스토리**를 토대로, 그 세계관의 캐릭터 **메르세데스**의 일대기를 다루는 액션 RPG를 Win32 API와 GDI+로 재현한 개인 포트폴리오다. 게임 엔진 없이 **1주일** 동안 기획부터 지형 이동 · 몬스터 AI · UI · 스킬 · 퀘스트까지 순서대로 붙여 나간 프로젝트다.

개발 환경	Visual Studio · Win32 Console/Windows Application · C++ · GDI+ · 1인 개발
사용 기술	GDI+ (이미지 로드) · GDI / msimg32 (TransparentBlt) · winmm (사운드)
개발 기간	7일 — 서울게임아카데미 1주 개인 포트폴리오 과제
장르	횡스크롤 액션 RPG (원작 '메이플스토리' 모작)
구현 범위	지형 이동 · 밧줄 · 몬스터 AI · 보스 · 스킬 5종 · UI · GDI 데미지 표기

일정 — 기획서에 적힌 7일 계획

- 1일차 — 이미지 추출, 기본 이동(좌우 · 점프), 화면 스크롤
- 2일차 — 몬스터 AI, 몬스터 사냥
- 3일차 — UI 구현 (GDI 데미지, 하단 UI, 인벤토리, 스킬창)
- 4일차 — 이동 구현 2단계(밧줄 · 충돌), 보스, 스킬 구현 1
- 5일차 — 퀘스트 3종, 스킬 구현 2
- 6일차 — 스킬 구현 3
- 7일차 — 테스트 · 버그 수정 · 동영상 제작

코드에는 이 순서가 그대로 남아 있다 — 이동과 충돌이 가장 촘촘하게 짜여 있고, 몬스터 AI와 스킬 시스템이 그 위에 얹혀 있으며, 퀘스트는 별도 클래스 없이 **보스를 잡으면 클리어**로 단순화돼 있다. 계획한 3단 퀘스트 체인 대신, 촉박한 일정 속에서 실제로 완성한 형태다.

2. 한 방향으로만 막는 발판 충돌

액션 플랫폼어의 발판은 **아래에서는 그대로 통과하고 위에서 떨어지면 멈춰야** 한다. 이 게임은 `FLOOR_TYPE` · `RECT_TYPE` · `ROPE_RECT_TYPE` 세 종류의 콜라이더로 지형을 구성하고, 그중 발판(`FLOOR_TYPE`)에 이 규칙을 적용했다.

그림 1. 위에서 밟고 아래에서 뚫는, 한 바닥의 두 가지 판정



왜 '이동량'이 아니라 '결과 위치'로만 판정하는가

중력 가속도 때문에 한 프레임의 낙하 이동량은 매번 다르다. 그 이동량으로 바닥을 뚫었는지 검사하면, 프레임이 조금만 밀려도 판정이 바닥을 통과해 오차가 난다. 그래서 이동 전/후 위치만 비교해 바닥을 지나쳤는지 보고, 지나쳤다면 그 프레임의 y 위치를 바닥면으로 강제 보정한다 — 이동 속도가 아무리 빨라도 뚫고 지나가는 순간이 생기지 않는다.

핵심은 **이동량이 아니라 이동 전/후의 결과 위치로 판정**한다는 점이다. 중력 가속도 때문에 한 프레임의 낙하 거리는 매번 달라지는데, 그 거리만큼 **발판을 뚫고 지나갔는지**를 매 프레임 검사하면 프레임이 조금만 밀려도 오차가 생겨 발판을 통과해버린다. 대신 발판을 **지나쳤는지 여부만 판정**하고, 지나쳤다면 그 프레임의 y 좌표를 발판면으로 강제 보정하는 방식을 썼다.

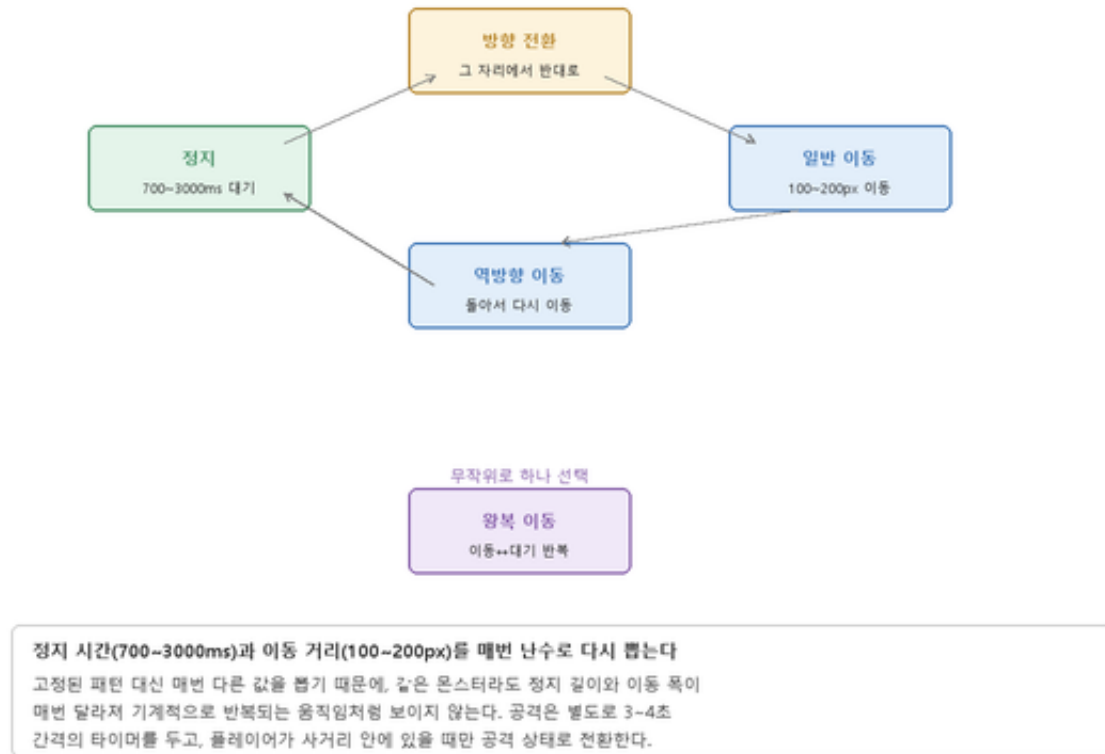
```
// 낙하 중 발판을 지나쳤는지 — 직전 위치와 발판, 지금 위치를 비교
if (latelyAirPosition.y <= mStartPosition->y &&
    nowPosition.y >= mStartPosition->y)
{
    // 뚫은 게 아니라, 그 프레임에 발판면으로 착지시킨다
    nowPosition.y = mStartPosition->y;
    return TRUE;
}
```

밧줄(ROPE_RECT_TYPE)도 같은 콜라이더 목록에 함께 들어 있지만 통행 규칙이 다르다 — 위/아래 방향키를 누르고 있는 동안만 그 영역 안에서 붙잡힌 상태로 상하 이동하고, 좌우 이동이나 점프 입력이 들어오면 즉시 놓게 했다. 발판과 밧줄이 같은 **Collider 클래스의 종류값만 다르게** 해서, 충돌 목록을 순회하는 코드는 한 종류만 신경 쓰면 되고 판정 함수만 종류별로 갈라진다.

3. 다섯 상태와 난수로 만드는 몬스터 AI

몬스터는 정지 · 방향 전환 · 일반 이동 · 역방향 이동 · 왕복 이동 다섯 가지 행동 상태를 두고, **정지 시간(700~3000ms)과 이동 거리(100~200px)를 매번 난수로 다시 뽑는다**. 고정된 순서로 상태를 도는 게 아니라 매번 무작위로 하나를 골라 전환하기 때문에, 같은 몬스터라도 다음에 어떻게 움직일지 예측하기 어렵다.

그림 2. 다섯 상태와 무작위 타이머로 만드는 몬스터 AI



공격은 별도의 3~4초 간격 타이머로 관리한다. 몬스터가 이동/정지 상태와 무관하게 이 타이머가 차고, 그 시점에 플레이어가 사거리 안에 있으면 공격 상태로 전환된다 — 즉 **이동 AI와 공격 판단이 서로 다른 시계로 독립적으로 돌아간다**. 하나의 타이머로 묶었다면 몬스터가 이동 중일 때는 공격을 못 하거나, 반대로 이동을 멈춰야만 공격하는 부자연스러운 패턴이 나왔을 것이다.

4. 스킬 하나 = 프레임 애니메이션 여러 개의 조합

플레이어는 원작 메르세데스의 스킬인 **여크로바틱 점프 · 레전더리 스피어 · 리프 토네이도 · 워터 실드 · 스피릿 인퓨전** 다섯 가지를 쓴다. 스킬 하나는 `mMotion` · `mSubMotion` · `mEffect` · `mHitEffect` · `mTileEffect` 다섯 개의 프레임 애니메이션을 한 구조체에 묶어 표현한다.

그림 3. 스킬 하나 = 프레임 애니메이션 여러 개의 조합



리프 토네이도는 이 조합이 왜 필요한지 잘 보여준다. 이 스킬은 `LTS_JUMP` → `LTS_ATTACK` 두 단계 상태를 가지고, 도약 중에는 `mMotion` 애니메이션이 재생되다가 착지와 동시에 `mSubMotion`(회전 낙하)으로 바뀌며 판정이 공격으로 전환된다. 하나의 스킬 입력이 서로 다른 두 개의 물리적 동작과 판정 구간으로 나뉘어 진행되는 구조라, 애니메이션을

한 개만 두면 이 전환을 표현할 수 없었다.

5. 숫자를 자릿수로 쪼개는 GDI 데미지 표기

타격 데미지는 텍스트가 아니라 숫자 이미지를 자릿수만큼 이어 붙이는 방식으로 그린다. `GetCipher()` 가 정수를 자릿수 단위로 분해하면, 각 자리에 해당하는 숫자 스프라이트를 왼쪽부터 나란히 그려 넣는다.

그림 4. 숫자를 자릿수로 쪼개 스프라이트로 합성하는 데미지 표기



자릿수 개수만큼 숫자 이미지를 하나씩 골라 나란히 그리고, 매 프레임 위로 떠오르며
알려지도록(alpha 감소) 좌표와 투명도를 함께 갱신한다.

`list<Damage>` 로 여러 개를 동시에 띄운다

몬스터가 짧은 시간에 연속으로 맞으면 이전 숫자가 아직 떠 있는 채로 새 숫자가 또
추가된다. 각 항목이 자기 위치 · 이동량 · 알파값을 따로 갖기 때문에 겹쳐도 문제없다.

데미지 숫자는 `list` 로 여러 개를 동시에 들고 있다가, 매 프레임 각자의 y좌표와 알파값을 갱신해 위로 떠오르며
열어지는 연출을 낸다. 몬스터가 짧은 시간에 연속으로 맞으면 이전 숫자가 아직 떠 있는 채로 새 숫자가 또
추가되는데, 각 항목이 자기 위치 · 이동량 · 투명도를 독립적으로 갖고 있어서 여러 개가 겹쳐도 서로 간섭하지
않는다.



실제 타격 장면. 노란 자릿수 스프라이트가 떠오르며 사라진다.

6. Find* 조회 패턴과 남은 문제

이미지 · 프레임 애니메이션 · 오브젝트 · 맵 · 몬스터를 모두 **Database** 싱글톤이 문자열 키로 보관하고,
`FindImageData` · `FindFrameAnimation` 같은 함수로 조회해 쓴다. 맵을 만들 때도 `MakeMap`(이름, 배경 애니메이션
키, 타일 이미지 키, ...) 처럼 리소스를 문자열 이름으로 등록하고 같은 이름으로 다시 찾아오는 구조라,
리소스 파일이 늘어나도 코드 쪽 수정 없이 키만 추가하면 된다.

```
Image* Database::FindImageData (string imageDataKey)
{
    return mImageData.find (imageDataKey)->second;
}

// 맵 등록도 같은 방식 — 이름 · 배경 키 · 타일 키를 문자열로 넘긴다
MakeMap ("습지", " ", tileImageKey, vCollider);
```

일주일짜리 프로토타입이 남긴 흔적

조회 실패를 검사하지 않는다. `find(key)->second` 는 키가 없으면 정의되지 않은 반복자를 그대로 반환한다. 문자열 키 몇 개가 저장 과정에서 깨져 있었는데, 처음 실행했을 때 이 지점에서 바로 걸렸다 — 시간에 쫓기며 짠 조회 코드가 정상 경로만 가정하고 있었다는 뜻이다.

퀘스트가 하드코딩된 클리어 조건으로 축소됐다. 기획서의 3단 퀘스트(사냥 → 파밍 → 보스) 대신, 실제 코드에는 별도의 퀘스트 클래스 없이 `PLAYER->GameClear()` 를 보스 처치 지점에 직접 호출하는 형태만 남아 있다. 일정 뒷부분(5~6일차)에 몰려 있던 항목이 가장 먼저 잘려나갔다.

7. 회고

이 프로젝트에서 가장 크게 배운 것은 **판정을 무엇 기준으로 세우느냐**의 차이였다. 발판 충돌을 이동량 기준으로 짰을 때는 프레임이 조금만 흔들려도 뚫고 지나가는 문제가 반복됐고, 기준을 **결과 위치 비교**로 바꾸고 나서야 속도와 무관하게 안정적으로 동작했다. 몬스터 AI에서도 이동과 공격을 **서로 다른 타이머로 분리**한 것이 같은 맥락이다 — 여러 판정이 한 기준을 억지로 공유하면 어느 한쪽이 부자연스러워진다는 걸 직접 겪었다.

일정 관리 면에서는 **뒤쪽 일정부터 잘려나간다는** 걸 몸으로 확인했다. 이동 · 충돌 · AI처럼 앞쪽에 배치한 시스템은 촘촘하게 완성됐지만, 퀘스트처럼 뒤쪽에 몰아둔 항목은 시간이 부족해지자 가장 단순한 형태로 축소됐다. 다음에는 **핵심 재미와 무관한 부가 시스템일수록 먼저 자를 수 있게** 앞쪽에 짧게라도 뼈대를 만들어 두는 편이 나을 것 같다.

코드 품질 면에서는 `Find*` 계열 함수 전체가 실패를 가정하지 않는다는 점이 가장 아쉽다. 1주일 프로토타입에서는 **정상 경로만 빠르게 짜는 것**이 합리적인 선택일 수 있지만, 리소스 키 하나만 어긋나도 프로그램 전체가 즉시 죽는 구조라는 것도 사실이다. 지금 다시 만든다면 조회 실패 시 로그를 남기고 안전하게 넘어가는 최소한의 방어 코드만큼은 처음부터 넣을 것 같다.

클래스 구성

`Database` 리소스 로드와 문자열 키 조회 (싱글톤) · `Object` 모든 게임 개체의 기반

`Player` 이동 · 밧줄 · 스킬 5종 · 상태 머신 · `Monster` 5상태 AI · 공격 타이머

`Collider` FLOOR / RECT / ROPE_RECT 3종 충돌 판정 · `Map` 지형과 배치된 오브젝트

`FrameAnimation` 이미지 시퀀스 재생 · `GraphicDamage` 자릿수 분해 데미지 표기

`StatusBar` HP/MP/보스 바 UI · `Warp` 맵 전환 · `Bullet` 원거리 판정



밧줄과 여러 층의 발판으로 구성된 던전 지형.